

Comparing Three Numerical Ways to Move a Two-Link Robot Arm

One-page project proposal

Base paper. D. Cardona-Ortiz and G. Arechavaleta, “Trajectory optimization for highly articulated robots based on sparsity-free local direct collocation,” *International Journal of Applied Mathematics and Computer Science*, 2025. <https://doi.org/10.61822/amcs-2025-0041>

In plain words: they plan motions of *large* robots by turning physics into a numerical problem. We reuse that idea on a *small* two-joint arm on a laptop. We will not rebuild their full industrial code.

Initial idea. We want a simple on-screen arm (two rods, two joints) to go from pose A to pose B. The arm exists only as equations. The computer can (1) guess motor commands and step physics forward in time, like a video game; (2) split time into a few points and connect them with a simple rule; or (3) use the same split, but a more accurate connecting rule (the style used in the base paper). We run all three on the *same* start, goal, and time budget, then compare who hits the target, who obeys physics better, and who is slower. A matplotlib animation will play the three arms side by side so the difference is visible.

What we will produce.

- Python code for the two-link arm and the three numerical recipes.
- A side-by-side animation (same clock, same target dot).
- A small table: miss distance, physics error, and runtime versus number of time points.

How this matches the course directions. *Cross-domain application:* we take course tools (stepping through time, connecting points, solving equations) and use them on a robot-arm motion problem, not on a textbook-only example.

Exploratory and comparative study: same arm and same task; three methods measured side by side on a simple model (not a claim of a new algorithm).

Out of scope. No real robot. No 6-joint industrial software from the base paper.

What the demo should look like (existing examples). Our animation will look like a two-rod drawing that moves, not a factory video. Look-alikes (not our code): Matplotlib double-pendulum animation and a two-link arm in Python (rparak). Our contribution is the three-method comparison, not a copy of those demos.